

# Vocabulary Builder MVP – Phase 1

## Engineering Troubleshooting Summary

### Project Context

Phase 1 focused on building the secure foundation of the Vocabulary Builder platform before introducing AI services such as Textract and Bedrock.

Core objectives:

- Secure website hosting
  - User authentication
  - Secure document storage
  - Infrastructure readiness for AI processing
  - Cost optimization
  - AWS security best practices
- 

## Case 1: CloudFront + OAC + Private S3 Website Access

### Service Involved

- Amazon S3
- Amazon CloudFront
- Origin Access Control (OAC)

### Problem

The website needed to be publicly accessible on the internet while keeping the S3 bucket completely private.

### Symptoms

After enabling S3 Block Public Access:

- Website stopped loading
- CloudFront returned Access Denied errors
- S3 objects became inaccessible

## Failed Troubleshooting Attempts

### Attempt 1

Making the S3 bucket public.

Result:

- Website worked
- Security requirements were violated

### Attempt 2

Using ACLs with public read permissions.

Result:

- Website worked
- Not aligned with modern AWS security best practices

## Root Cause

CloudFront did not have permission to access objects stored inside the private S3 bucket.

## Final Solution

Implemented:

- CloudFront Origin Access Control (OAC)
- SigV4 request signing
- S3 bucket policy allowing only CloudFront access

Architecture:

User

→ CloudFront

→ OAC (SigV4 Signed Request)

→ Private S3 Bucket

## Engineering Lesson Learned

A public website does not require a public S3 bucket.

Modern AWS best practice is:

Private S3 + CloudFront + OAC

---

## Case 2: BucketOwnerEnforced Migration

### Service Involved

- Amazon S3

### Problem

Permissions were becoming difficult to manage when multiple authorization systems existed simultaneously.

### Symptoms

Potential permission conflicts between:

- ACLs
- IAM Policies
- Bucket Policies

### Root Cause

Multiple permission models create troubleshooting complexity and increase the risk of configuration errors.

### Final Solution

Enabled:

BucketOwnerEnforced

Result:

- ACLs disabled completely
- Permissions managed only through IAM and Bucket Policies

## Engineering Lesson Learned

Reducing authorization layers simplifies troubleshooting and improves security.

---

# Case 3: Single Bucket vs Two-Bucket Architecture

## Service Involved

- Amazon S3

## Problem

Deciding whether website files and uploaded documents should share the same bucket.

## Initial Option

Single bucket structure:

- website/
- documents/
- outputs/

## Risks Identified

Misconfigured permissions could expose:

- User PDFs
- OCR results
- AI-generated content

## Final Solution

Separated responsibilities:

Website Bucket

vb-website-2025

Documents Bucket

vb-documents-2025

## Engineering Lesson Learned

Workload isolation reduces operational and security risk.

---

# Case 4: Secure Document Storage Design

## Service Involved

- Amazon S3

## Problem

User-uploaded documents required secure long-term storage.

## Security Controls Implemented

### Block Public Access

Enabled

### Encryption

SSE-S3 Enabled

### Versioning

Enabled

## Benefits

- Protection against accidental deletion
- Encryption at rest
- Strong security baseline

## Engineering Lesson Learned

Security should be designed into the architecture from the beginning, not added later.

---

# Case 5: Lifecycle Cost Optimization

## Service Involved

- Amazon S3

## Problem

OCR and AI processing outputs would continuously increase storage costs.

## Solution

Lifecycle policies:

raw/

→ Delete after 90 days

textract-output/

→ Delete after 730 days

comprehend-output/

→ Delete after 730 days

## Engineering Lesson Learned

Cost optimization is an architectural responsibility, not an operational afterthought.

---

## **Case 6: Cognito Authentication Architecture**

### **Service Involved**

- Amazon Cognito

### **Problem**

The platform required secure authentication without building a custom identity system.

### **Rejected Approach**

Custom authentication implementation.

Challenges:

- Password management
- Password hashing
- MFA
- Email verification
- Password reset flows
- Session management

### **Final Solution**

Implemented:

- Amazon Cognito User Pool
- Hosted UI
- OAuth2 Authorization Code Flow
- PKCE

### **Engineering Lesson Learned**

Use managed identity services whenever possible instead of building authentication systems from scratch.

---

## Key Phase 1 Achievements

- ✓ CloudFront configured and operational
  - ✓ HTTPS enabled
  - ✓ Private S3 website hosting implemented
  - ✓ OAC configured successfully
  - ✓ Cognito authentication working
  - ✓ Secure document storage established
  - ✓ Encryption enabled
  - ✓ Versioning enabled
  - ✓ Lifecycle policies configured
  - ✓ Infrastructure prepared for Phase 2 AI services
- 

## Most Valuable Engineering Story

One of the most important challenges during Phase 1 was serving a public website while keeping the S3 bucket completely private.

Initially, making the bucket private caused CloudFront access failures.

After analyzing the request path, the issue was traced to missing trust between CloudFront and S3.

The solution was to implement Origin Access Control (OAC), which uses SigV4-signed requests. This allowed S3 to verify that requests originated only from CloudFront.

The result was a secure architecture that maintained public website availability without exposing the S3 bucket directly to the internet.

This became one of the most valuable architecture and security lessons of the project.